

Log Processor | Part I

CMPE 202 · Design write-up and class diagram

Name	Kalhar Mayurbhai Patel
SJSU ID	019140511
Project	Log Processor (CLI parser and aggregator)
Stack	Java 11, Maven, JUnit 5, Jackson

1. Problem Being Solved

Modern systems generate large volumes of mixed log data on a single stream: application performance metrics (CPU, memory, disk and other telemetry), application events (errors, warnings, info, debug), and HTTP request traces. Reading these manually is slow and error-prone, and a single file mixes all three formats together.

The goal of this project is a command line tool that:

- reads a plain text log file passed as `--file <name.txt>`;
- classifies each line as APM, Application or Request, and silently ignores corrupted or unknown lines;
- computes the required aggregations per category (min, median, average, max for APM metrics; counts per severity for application logs; min, 95th percentile, max response time and 2XX / 4XX / 5XX counts per route for request logs);
- writes `apm.json`, `application.json` and `request.json`, even when no logs of that type were seen;
- is open to new log types and new output formats without changing existing classes.

2. Design Patterns Used

The solution uses three complementary patterns. Each addresses a specific concern: how to dispatch lines to the right parser, how to compute statistics per log type, and how to share common parsing behaviour across all handlers.

A. Chain of Responsibility (primary)

Because a single file mixes APM, Application and Request lines, the type of a line is not known up front. Each log type has its own handler that extends the abstract `LogHandler`. The handlers are linked with `setNext(...)`. Every line is sent to `handle(...)` on the head of the chain; if a handler claims the line, processing stops, otherwise the line is forwarded to the next handler. Lines that no handler claims fall off the end and are dropped, which implements the rule for corrupted input without any branching at the call site.

B. Strategy (aggregation)

Each log type needs a different aggregation algorithm: averages and percentiles for numeric metrics, counts for severity levels, and per-route bucketing for HTTP requests. These algorithms are encapsulated in three aggregator classes, `ApmAggregator`, `ApplicationAggregator` and `RequestAggregator`, each with the same shape: `add(...)` records an observation and `getAggregated()` returns a JSON ready map. Handlers parse and dispatch; aggregators compute. New aggregation algorithms can be added without touching the handlers.

C. Template Method (handler skeleton)

All handlers share the same control flow: try to parse the line, and forward it to the next handler if it does not match. This skeleton lives in `LogHandler.handle(...)`. Subclasses only implement the variable step,

`process(line)`, which returns true if the line was claimed. Null and blank lines are filtered once in the base class, so subclasses stay focused on parsing.

3. Consequences of the Chosen Patterns

A. Chain of Responsibility

Pros	Cons
• Flexible flow: handler order can be changed or replaced at runtime. • Decoupling: handlers are independent and unaware of each other. • Robust input handling: unknown or corrupted lines are skipped without crashing. • Open / Closed: new log types are added by inserting a new handler, not by editing existing code.	• Performance overhead: an unrecognised line is checked by every handler before being dropped ($O(n)$ per entry). • Debugging difficulty: it is not always obvious which handler ultimately claimed a line. • No guaranteed match: lines that no handler recognises are silently ignored, which has to be an explicit design choice.

B. Strategy

Pros	Cons
• Extensibility: new aggregations are added by writing a new class, not by editing existing ones. • Single responsibility: each aggregator owns one algorithm and one data structure. • Testability: aggregators can be unit tested in isolation, without any file or chain setup. • Swappability: handlers depend on the aggregator role, not on a concrete class.	• Class proliferation: one extra class per log type. • Higher complexity: may be heavier than a few helper methods for very small projects. • Learning curve: developers need to know which aggregator pairs with which handler.

C. Template Method

Pros	Cons
• Code reuse: the chain walking and null / blank filtering are written once in the base class. • Consistent behaviour: every handler follows the same control flow, which removes a class of bugs. • Focused subclasses: concrete handlers contain parsing logic only.	• Inheritance coupling: subclasses are tied to the base class shape; large changes in the base ripple downward. • Hidden flow: the order of operations lives in the base class, which can be surprising at first.

Net effect. Together the three patterns separate concerns cleanly: **Chain of Responsibility** picks the right handler at runtime, **Strategy** isolates each aggregation algorithm, and **Template Method** keeps the chain walking logic in one place. Adding a fourth log type means writing one handler subclass and one aggregator class, and inserting one `setNext` call. No existing class needs to change.

4. Class Diagram

Concrete handlers extend LogHandler and each owns one aggregator. The dashed arrows show creation and delegation. The yellow note on the right summarises the chain behaviour. PlantUML source: docs/ClassDiagram.puml.

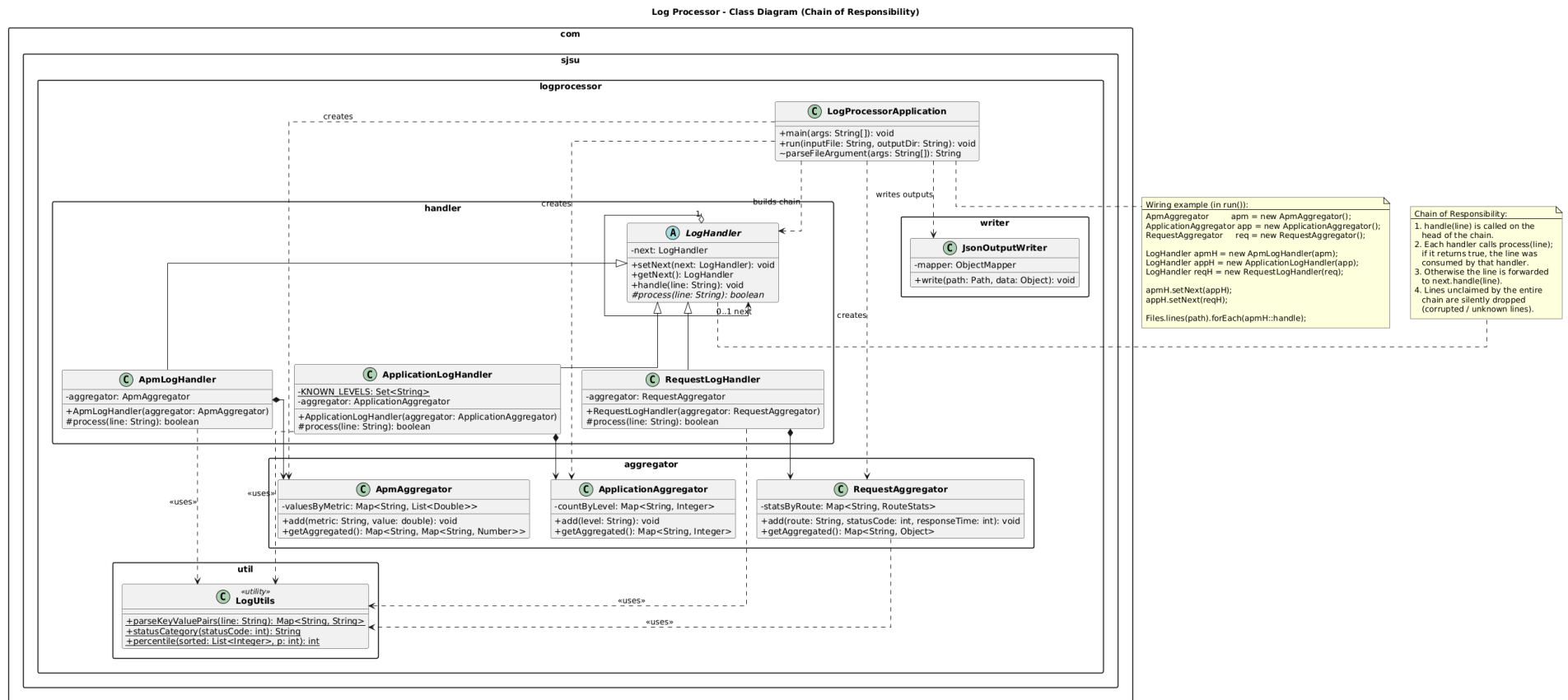


Figure 1. Class diagram. Chain of Responsibility, Strategy aggregators, Template Method base.